13.6 Database Connection Pooling

Topic 6 of 15 | ~5 hours | Difficulty: Intermediate

1 What it is

Every client of a PostgreSQL server holds a **connection**, and each connection is a dedicated operating-system **process** (a "backend") with its own memory, started by the server when you connect. Opening one costs a TCP handshake, usually a TLS handshake, authentication and a process fork, which is milliseconds to tens of milliseconds, and keeping one open costs several megabytes of RAM even while it does nothing. The server also has a hard ceiling, `max_connections` (default 100).

Connection pooling puts a small, reusable set of already-open database connections between your application and the database. There are two layers and you usually want both:

- **In-app pool** (HikariCP in Java, `pg.Pool` in Node.js, SQLAlchemy's pool in Python): each application process keeps N connections open and lends them to request handlers.
- **External pooler** (**PgBouncer**, Pgpool-II, Odyssey, PgCat, Amazon RDS Proxy): a separate lightweight proxy that accepts thousands of client connections and multiplexes them onto a few dozen real server connections. This is what lets 50 application instances share one database without exhausting it.

The central idea: **a database is fastest with a small number of busy connections, not a large number of idle ones.**

2 Why it matters in DevOps

- **"FATAL: sorry, too many clients already"** (or `remaining connection slots are reserved for roles with the SUPERUSER attribute`) is a top-five production Postgres error. It tends to appear exactly when traffic spikes or you scale the app out, i.e. when you can least afford it.
- **Autoscaling multiplies connections.** 20 app pods x a pool of 10 = 200 connections. Kubernetes happily scales to 40 pods during a sale; the database does not scale with them. See High Availability and Scalability §0.8.

- **Serverless and short-lived processes** (AWS Lambda, cron jobs, CI jobs) open a fresh connection per invocation, which is the worst case for a process-per-connection database.
- **More connections can mean *less* throughput.** Past a point, hundreds of active backends fight for CPU, memory and locks, and latency rises for everyone. A pool that *queues* excess work beats a database that *thrashes*.
- **Operational leverage:** a pooler lets you restart or fail over the database while clients stay connected (`PAUSE` / `RESUME`), cap each service's connections, and see who is using the database. Related: Connection Pooling §12.10 in the networking level covers the general idea; this topic is the PostgreSQL-specific case.

3 Real-world system example

Amazon RDS Proxy is AWS's managed pooler for RDS and Aurora (including PostgreSQL). Its documented purpose is exactly the one in this topic: pool and share connections to reduce database load, keep connections from Lambda functions and other bursty clients from overwhelming the database, and shorten failover time by holding client connections while the database fails over. See RDS and Aurora §6.10.

GitLab ships PgBouncer as a component of its Linux package for PostgreSQL deployments and documents running it in front of PostgreSQL (with Patroni for failover), because the Rails application would otherwise open far too many connections. **Heroku Postgres** offers a connection-pooling add-on built on PgBouncer in transaction mode, and **Supabase** documents transaction-mode pooling (via its own pooler, Supavisor) for serverless and edge clients. In every one of these, the application keeps hundreds or thousands of client connections while the database sees a few dozen.

PgBouncer is the de facto standard because it is tiny: a single-threaded C process using an event loop, a few KB per client connection, and a simple text admin console.

4 Diagram

Many clients, few server connections:

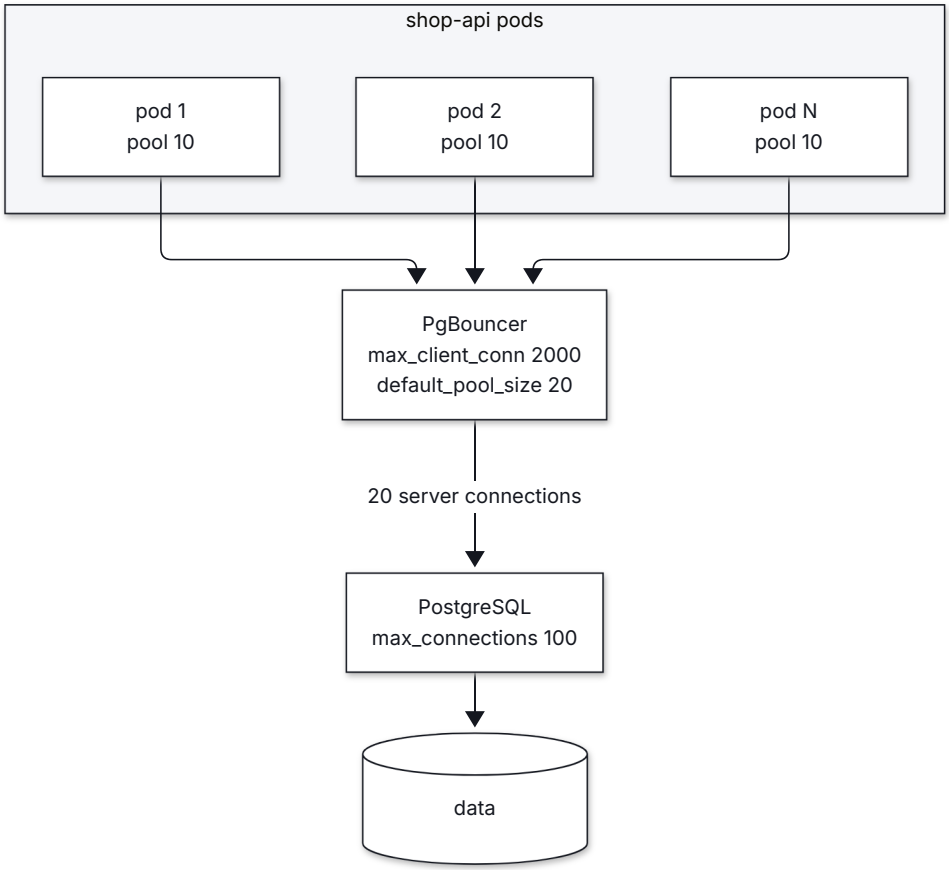


Figure 13.6-1

What transaction pooling does over time. The server connection is borrowed for one transaction and returned straight away:

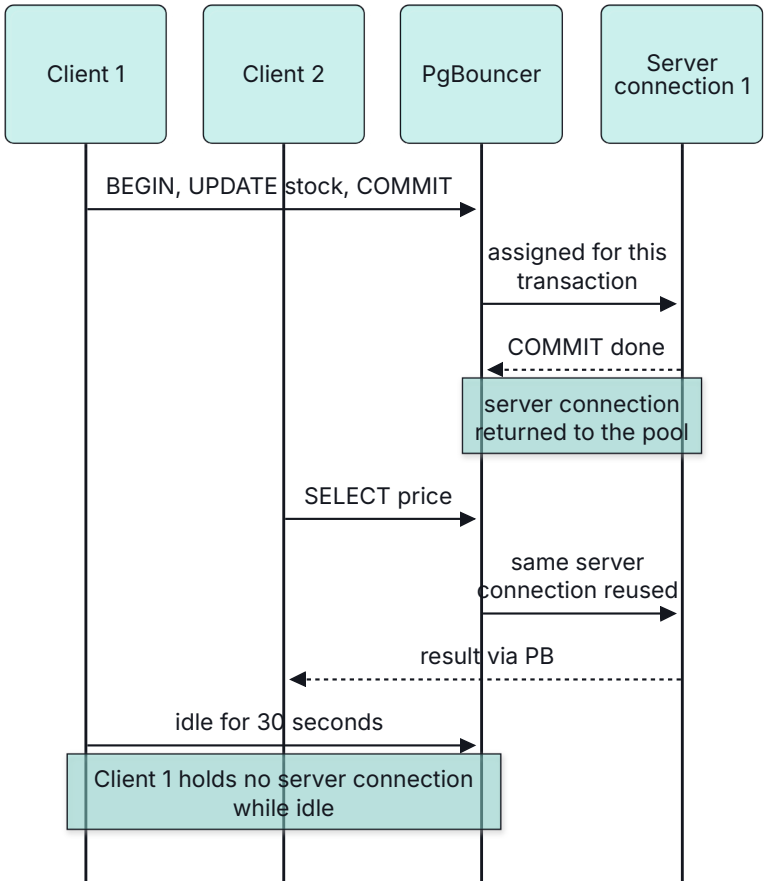


Figure 13.6-2

5 How it works

Why connections are expensive

- Each backend is a process. PostgreSQL has per-backend memory (catalog caches, plans) and can use up to `work_mem` per sort/hash node on top. Hundreds of connections multiply this.
- Every active backend takes a slot in shared structures (the process array, lock tables), and snapshot-taking scans them; very large connection counts add measurable overhead even when most are idle.

- Setup cost: TCP + TLS + authentication (SCRAM) + fork. A short-lived client that connects per query can spend more time connecting than querying.

PgBouncer pool modes

Mode	Server connection is held for	Good for	Breaks
session	the whole client connection	legacy apps that use session state; a safe default	does not reduce idle connections much; each long-lived client pins one server connection
transaction	one transaction	web/API apps and serverless: the big win	anything that relies on session state (see below)
statement	one statement	autocommit-only workloads	multi-statement transactions are not allowed

Transaction mode is the one you want for `shop-api`, and it comes with rules. Between transactions the next one may run on a *different* server connection, so session-level state can be wrong, lost, or leak to another client:

Feature	Problem in transaction mode	What to do
`SET x = ...` (session-level)	stays on the server connection and **leaks** to the next client	use `SET LOCAL` inside a transaction, or set per-role/per-database defaults in the server
SQL-level `PREPARE` / `EXECUTE`	the statement exists only on one backend: "does not exist" or "already exists" errors	use PgBouncer 1.21 or newer with `max_prepared_statements` set (it tracks protocol-level prepared statements), or disable prepared statements in the driver
Session advisory locks (`pg_advisory_lock`)	lock belongs to a server connection that gets reused	use transaction-level locks `pg_advisory_xact_lock`
`LISTEN` / `NOTIFY`	notifications need a stable session	connect that component directly to the database
`WITH HOLD` cursors, temp tables across transactions	tied to a session	keep inside one transaction

The numbers: `max_connections` and pool sizing

PostgreSQL side:

- `max_connections` (default 100) is the total number of backends. `superuser_reserved_connections` (default 3) are kept for admins, so ordinary users see a lower effective limit. Increasing it needs a restart and raises memory use.

- Replication, `pg_basebackup`, autovacuum workers and monitoring agents also count or compete for slots.

PgBouncer side (the settings you will touch):

Setting	Meaning
`pool_mode`	`session`, `transaction` or `statement`
`default_pool_size`	server connections per (database, user) pair (default 20)
`max_client_conn`	how many client connections PgBouncer accepts (default 100; raise it to thousands; limit by file descriptors)
`max_db_connections` / `max_user_connections`	hard cap on server connections per database or user across pools
`min_pool_size`, `reserve_pool_size`, `reserve_pool_timeout`	keep warm connections; allow a burst of extra connections after waiting this long
`query_wait_timeout`	how long a client may wait for a server connection before being dropped (default 120 s)
`server_idle_timeout`, `server_lifetime`	recycle server connections
`auth_type`, `auth_file`, `auth_query`	how clients authenticate (`scram-sha-256` is the modern choice; `auth_query` avoids maintaining a user file)
`server_reset_query`	cleanup run when a session-mode connection is returned (`DISCARD ALL` by default; not run in transaction mode unless configured)

Sizing math. Start from how much work the database can really do in parallel, and let everything else queue.

1. **Server-side budget.** A common starting point (from the PostgreSQL community wiki and HikariCP's pool-sizing guidance) is `connections ≈ (CPU cores x 2) + effective_spindle_count`. For an SSD-backed 8-core database that is about 16-20 *active* connections. Treat it as a starting point and load-test; more connections than this rarely add throughput for CPU-bound work.
2. **Little's law.** The number of connections busy at once is `throughput x time each request holds a connection`. `shop-api` at 1,500 requests/s where each request holds a connection for 4 ms of DB work needs about `1500 x 0.004 = 6` connections on average. Size for peaks (say 3x), so 18-20, not 200.
3. **Check the fan-out.** Without a pooler: `pods x pool_size_per_pod` must fit under `max_connections - reserved - other_clients`. `20 pods x 10 = 200 > 100`, so the database refuses connections. With PgBouncer: apps may open 20 x 10 = 200 *client* connections, but `default_pool_size = 20` keeps the database at 20.

4. **Multiple pools multiply.** PgBouncer pools are per (database, user). Three services with three roles and `default_pool_size = 20` can open 3 x 20 = 60 server connections. Add `max_db_connections` so the sum stays inside the budget.
5. **Reserve headroom.** Leave 10-20 slots for admins, replication, migrations and monitoring.
6. **Make the in-app pool small too.** An app pool of 10 per pod that *waits* for a PgBouncer connection is fine. Set app-side connection and acquire timeouts shorter than your request timeout so waiting does not pile up.

The queueing behaviour is the feature: when 200 clients ask for 20 connections, 180 wait milliseconds in PgBouncer's queue instead of making Postgres thrash. Watch `cl_waiting` and `maxwait` in `SHOW POOLS`; if they stay high you are under-provisioned or have slow queries.

6 Step-by-step guide

Prerequisites: a Linux machine (or WSL2), Docker (for the database), and `sudo apt-get install pgbouncer postgresql-client` on Debian/Ubuntu. Free, local, nothing in the cloud. The outputs below were taken from PostgreSQL 16 and PgBouncer 1.22; yours will be "similar to" these.

1. **Start a PostgreSQL with a deliberately low connection limit.**

```bash
docker run -d --name pg-pool -e POSTGRES_PASSWORD=pgadmin -p 5455:5432 postgres:16 -c
  max_connections=20
sleep 5
psql "postgresql://postgres:pgadmin@127.0.0.1:5455/postgres" -c "create role shop
  login password 'shop_pw'" -c "create database shop owner shop"
```

   Expect `CREATE ROLE` and `CREATE DATABASE`. `show max_connections;` returns `20`.

2. **Create sample data and hit the limit directly.**

```bash
export PGPASSWORD=shop_pw
pgbench -h 127.0.0.1 -p 5455 -U shop -i -s 1 shop
pgbench -h 127.0.0.1 -p 5455 -U shop -c 50 -j 2 -T 5 shop
```

(`pgbench` is in the `postgresql-contrib` or `postgresql-16` packages depending on the distro.) 50 clients cannot connect to a server that allows 20, so you should see something like:

```
pgbench: error: connection to server at "127.0.0.1", port 5455 failed: FATAL:
  remaining connection slots are reserved for roles with the SUPERUSER attribute
pgbench: error: could not create connection for client 8
```

That is the outage, reproduced.

3. **Configure PgBouncer in transaction mode.**

```bash
mkdir -p ~/pgb && cd ~/pgb
cat > pgbouncer.ini <<'EOF'
[databases]
shop = host=127.0.0.1 port=5455 dbname=shop

[pgbouncer]
listen_addr = 127.0.0.1
listen_port = 6432
auth_type = scram-sha-256
auth_file = /home/YOU/pgb/userlist.txt
admin_users = shop
pool_mode = transaction
default_pool_size = 5
max_client_conn = 200
reserve_pool_size = 2
reserve_pool_timeout = 3
query_wait_timeout = 30
logfile = /home/YOU/pgb/pgbouncer.log
pidfile = /home/YOU/pgb/pgbouncer.pid
unix_socket_dir = /home/YOU/pgb
EOF
echo '"shop" "shop_pw"' > userlist.txt && chmod 600 userlist.txt
sed -i "s#/home/YOU#$HOME#g" pgbouncer.ini
pgbouncer -d pgbouncer.ini
```

`userlist.txt` holds a plaintext password only because this is a lab. In production use `auth_query` (so PgBouncer looks users up in the database) or SCRAM verifiers, and TLS between the layers. Note `admin_users = shop` also makes `shop` an admin of the pooler, which a real deployment would not do.

4. **Run the same load through the pooler.**

```bash
pgbench -h 127.0.0.1 -p 6432 -U shop -c 50 -j 2 -T 5 shop
```

Expect `number of failed transactions: 0 (0.000%)` and a `tps` in the low thousands on a laptop. 50 clients succeeded against a 20-connection database.

5. **Look inside the pooler while it works.** Run the load in the background and open the admin console (the special database `pgbouncer`):

```bash
pgbench -h 127.0.0.1 -p 6432 -U shop -c 50 -j 2 -T 20 shop > /dev/null &
sleep 3
psql -h 127.0.0.1 -p 6432 -U shop pgbouncer -c "SHOW POOLS" -c "SHOW STATS" -c "SHOW
  SERVERS"
psql -h 127.0.0.1 -p 5455 -U shop shop -tAc "select count(*) from pg_stat_activity
  where usename='shop' and backend_type='client backend'"
```

In `SHOW POOLS` for `shop` you should see roughly `cl_active=5`, `cl_waiting=45` and `sv_active=5`: **50 clients are served by 5 server connections** with the rest queued. The `pg_stat_activity` count (about 5, plus this query) confirms PostgreSQL sees only those. `maxwait` shows how long the longest-waiting client has waited.

6. **See the transaction-mode trap (session state leaking).**

```bash
psql -h 127.0.0.1 -p 6432 -U shop shop -c "SET statement_timeout='7s'" -c "SELECT
  pg_backend_pid()"
psql -h 127.0.0.1 -p 6432 -U shop shop -c "SHOW statement_timeout"  -c "SELECT
  pg_backend_pid()"
```

Two different client sessions, the **same backend PID** and the second one sees `7s` set by the first. Likewise `PREPARE q AS SELECT 1` from two separate clients fails the second time with `prepared statement "q" already exists`. These are the bugs people hit when they switch a legacy app to transaction mode. The fix is to use `SET LOCAL`, drop SQL-level `PREPARE`, or (PgBouncer 1.21 and newer) set `max_prepared_statements = 100` so protocol-level prepared statements from drivers are handled; check your driver's documentation.

7. **Provoke pool exhaustion on purpose.** Set `default_pool_size = 1`, `reserve_pool_size = 0` and `query_wait_timeout = 2`, then reload:

```bash
psql -h 127.0.0.1 -p 6432 -U shop pgbouncer -c "RELOAD"
psql -h 127.0.0.1 -p 6432 -U shop shop -c "select pg_sleep(5)" &
sleep 0.5; psql -h 127.0.0.1 -p 6432 -U shop shop -c "select 1"
```

The second client waits about two seconds and gets `FATAL: query_wait_timeout`. That error in your application logs means the pool is too small for the queue, or a few slow queries are holding every connection. Put your settings back afterwards.

8. **Graceful maintenance with PAUSE/RESUME.**

```bash
psql -h 127.0.0.1 -p 6432 -U shop pgbouncer -c "PAUSE" -c "RESUME"
```

`PAUSE` waits for in-flight transactions to finish and holds new ones; you can restart or fail over the database, then `RESUME`. Clients see latency rather than errors.

9. **Clean up.**

```bash
kill "$(cat ~/pgb/pgbouncer.pid)"
docker rm -f pg-pool
```

7 Real-world scenario

(Illustrative, not a documented incident.) `shop-api` runs 12 pods, each with a Node.js `pg.Pool` of `max: 10`, against a PostgreSQL with `max_connections = 100`. Normal traffic is fine. During a Friday flash sale the HPA scales the API to 30 pods in four minutes. Suddenly 30 x 10 = 300 potential connections exceed 100 and the error logs fill with `sorry, too many clients already`. Checkout fails; the on-call engineer sees healthy pods but a database whose CPU is also pinned at 100% by hundreds of runnable backends.

Investigation: `select count(*), state from pg_stat_activity group by state` shows 97 connections, most `idle` (pods holding pools open) and a few `active`. The database is not busy; it is crowded. The immediate fix is to cap the HPA and restart the slowest queries; the permanent fix is a PgBouncer deployment in transaction mode between the pods and the database with `default_pool_size = 20` and `max_client_conn = 2000`, plus smaller per-pod pools. They first audit the code for `SET` statements and advisory locks, find one `pg_advisory_lock` in the order-numbering code, and change it to `pg_advisory_xact_lock`.

At the next sale 40 pods open 400 client connections; PostgreSQL sees 20 server connections, CPU is healthy, and `cl_waiting` peaks at a few dozen for milliseconds. **Lesson:** the database's concurrency budget is fixed, so fan-out must be absorbed by a queue (the pooler), and "works at 12 pods" says nothing about 40. Test the transaction-mode assumptions before the incident, not during it.

8 Common mistakes and best practices

Don't

- Raise `max_connections` to 1000 as the first response. It uses more memory and usually makes latency worse; it only postpones the problem.
- Switch to transaction mode without auditing session state (`SET`, session advisory locks, `PREPARE`, `LISTEN`, temp tables).
- Let every service create its own large pool "just in case". Totals add up across pods, pools and users.
- Point migrations, `pg_dump`, logical replication or `LISTEN` consumers at the transaction pool. Give admin and batch work a direct (or session-mode) path.
- Expose PgBouncer without TLS and authentication, or store production passwords in a plaintext `userlist.txt`.
- Forget that the pooler is now a critical component: run more than one behind a load balancer (it is cheap) and monitor it.

Do

- Size pools from the database's real capacity and Little's law, then confirm with a load test; prefer a small pool plus a queue.
- Use transaction mode for web/API/serverless workloads, session mode where you need session features, and keep a direct admin connection.
- Set timeouts on both sides (`query_wait_timeout`, app acquire timeout, `statement_timeout`, `idle_in_transaction_session_timeout`) so waits do not cascade.
- Alert on `cl_waiting`, `maxwait`, pool utilization and `FATAL: ... too many clients` in logs; export PgBouncer metrics (for example with a Prometheus exporter, see Prometheus and PromQL §10.2).
- Use `auth_query` and TLS in production, and plan for failover with `PAUSE` / `RESUME` or a managed pooler such as RDS Proxy.

9 Tasks

- [ ] **T1 — Reproduce "too many clients"** *(≈25 min, beginner)*
  - **Goal:** see the failure that connection pooling prevents.
  - **Steps:**
    1. Start Postgres in Docker with `-c max_connections=20` (step 1 of the guide).
    2. Run `pgbench -c 50 -T 5` directly against it.
    3. In a second terminal during a smaller run (`-c 15`) count connections: `select count(*), state from pg_stat_activity group by state;`.

- **Done when:** you have captured the exact `FATAL` message and can explain why 15 clients worked but 50 did not (including the reserved superuser slots).

☐ **T2 — Put PgBouncer in front and watch it multiplex** *(≈40 min, beginner)*

- **Goal:** serve many clients with few server connections.
- **Steps:**
  1. Follow guide steps 3-5 with `default_pool_size = 5`.
  2. During a 50-client run record `cl_active`, `cl_waiting` and `sv_active` from `SHOW POOLS`.
  3. Compare against the `pg_stat_activity` count from Postgres' side.
- **Done when:** you have numbers showing 50 client connections mapped to about 5 server connections and zero failed pgbench transactions.

☐ **T3 — Find the best pool size by experiment** *(≈60 min, intermediate)*

- **Goal:** replace guesswork with a measured number.
- **Steps:**
  1. With `-c 50`, run pgbench for 20 s each at `default_pool_size` of 1, 2, 4, 8, 16 (edit and `RELOAD`; remember `max_connections=20`).
  2. Record `tps` and average latency for each in a table.
  3. Compare the knee of the curve to `(CPU cores x 2)` for your machine, and write down how the answer would change for a database with slower disk or longer queries.
- **Done when:** you have a table of pool size vs tps/latency and a one-paragraph conclusion naming your chosen size.

☐ **T4 — Break transaction mode and fix it** *(≈45 min, intermediate)*

- **Goal:** experience session-state bugs safely.
- **Steps:**
  1. Reproduce the `SET` leak and the `PREPARE ... already exists` error from guide step 6.
  2. Rewrite the first as `BEGIN; SET LOCAL statement_timeout='7s'; ...; COMMIT;` and prove the next client does not see it.
  3. Replace the session advisory lock with `pg_advisory_xact_lock` inside a transaction and verify it is released at commit.
- **Done when:** a second client always sees the default `statement_timeout` after your fixed version, and you can list three features that need session mode or a direct connection.

- [ ] **T5 — Provoke and alert on pool exhaustion** *(≈45 min, intermediate)*
  - **Goal:** recognise and monitor the saturation signal.
  - **Steps:**
    1. Set `default_pool_size = 1` and `query_wait_timeout = 2`, then hold the only connection with `select pg_sleep(30)`.
    2. Make another client time out with `FATAL: query_wait_timeout`.
    3. Write a small script that runs `SHOW POOLS` every 5 s and prints a warning if `cl_waiting > 0` for more than two checks or `maxwait > 1`.
  - **Done when:** your script prints a warning while the slow query runs and stays quiet when you remove it.

- [ ] **T6 — Prove it: size the pools for a scale-out** *(≈45 min, advanced)*
  - **Goal:** produce a defensible sizing plan without the guide.
  - **Steps:**
    1. Given: 8-core database, `max_connections=200` but you want at most 30 active, 3 services (`shop-api`, `worker`, `reports`) with separate DB roles, `shop-api` scales 5 to 50 pods, peak 3,000 req/s at 5 ms DB time per request.
    2. Calculate the required concurrent connections with Little's law, choose `default_pool_size`, `max_db_connections`, `max_client_conn` and per-pod app pool size, and write the PgBouncer config.
    3. Explain in writing what happens when all 50 pods are busy and the pool is full, and which timeout fires first.
  - **Done when:** your plan shows the math (3,000 x 0.005 = 15 average, peak headroom), the server connections stay inside the budget, and a teammate can follow your reasoning.

10 Self-check questions

1. Why is a PostgreSQL connection more expensive than a connection to, say, an HTTP server?
2. Your 40 pods each hold a pool of 10 and PostgreSQL has `max_connections = 100`. What happens, and what are two fixes?
3. Which PgBouncer pool mode do you use for an API, and what is the main risk?
4. What does `cl_waiting > 0` in `SHOW POOLS` tell you?
5. Roughly how many connections should a database with 8 cores keep active, and why not 500?

6. Why can `pg_advisory_lock` be dangerous behind a transaction pooler?

ANSWERS

1. Each connection is a dedicated server process with its own memory, created by a fork plus authentication and TLS; the server also has a fixed limit of backends. HTTP servers typically multiplex many connections in a few threads.
2. 400 potential connections exceed 100, so new connections are refused with "too many clients" (and the rest of the DB is crowded). Fixes: put a pooler such as PgBouncer in transaction mode in front, shrink the per-pod pool, cap pod count, or all three. Raising `max_connections` is a last resort.
3. Transaction mode. The risk is session state (`SET` , session advisory locks, `PREPARE` , `LISTEN` , temp tables) breaking or leaking because consecutive transactions may use different server connections.
4. Clients are queued waiting for a server connection; the pool is saturated. A brief spike is the pooler doing its job; sustained waiting means pool too small or slow queries holding connections, and eventually `query_wait_timeout` errors.
5. A starting point is (cores x 2) plus disk parallelism, so roughly 16-20, then load test. More active backends than the hardware can run in parallel only adds contention and context switching and increases latency.
6. The lock belongs to a server connection, and the pooler may give that connection to another client or give you a different one later, so you may unlock on the wrong connection or leave the lock held and block others. Use `pg_advisory_xact_lock` inside a transaction.

11 Further reading

- PgBouncer documentation (pgbouncer.org): the configuration reference and the *Features* page, which lists exactly which SQL features work in each pool mode.
- PostgreSQL documentation: *Server Configuration - Connections and Authentication* (`max_connections` , `superuser_reserved_connections`) and the `pg_stat_activity` view.
- PostgreSQL wiki: *Number of Database Connections*, and the HikariCP wiki page *About Pool Sizing*.
- Amazon RDS User Guide: *Amazon RDS Proxy*.
- *PostgreSQL 14 Internals* (Egor Rogov, free PDF) for what a backend process holds.
- Related: Replication §13.4, Horizontal and Vertical Scaling §13.10.